

WOXSEN UNIVERSITY

DBMS Course Project Report

Design and Implementation of a Database Management System for Water Tanker Booking and Delivery Management System

Item	Details
Course	Database Management Systems (DBMS)
Submitted by	T Chandrahas (Chandrahas Thunga)
Roll number	25WU0102284
Programme and section	B.Tech (AIML), 2nd Year, Semester III, AIML Whales
University	Woxsen University
Faculty	Dr. Kiranmayee Adavala
Academic year	2026-2027
Date of submission	07.10.2026

1. Abstract

This project delivers a normalized MySQL database and a working web application for managing water tanker bookings and deliveries. Water tanker supply is still booked mostly by phone and paper registers, which causes lost orders, double-booked vehicles, blind dispatching and disputed cash payments. The database, named `water_tanker_db`, has nine tables in Third Normal Form (CUSTOMER, AREA, ADDRESS, TANKER_TYPE, TANKER, DRIVER, BOOKING, DELIVERY and PAYMENT), one reporting view (`vw_booking_details`) and three triggers. The triggers enforce the business rules inside the database: a tanker cannot be double-booked, its size must match the booking, a driver cannot be on two deliveries at once, and booking and tanker statuses update automatically. A React web application named AquaFlow sits on top of the database and supports inserting, deleting and viewing records, dispatching tankers, and tracking revenue. This report documents the problem, the ER model, the schema and normalization, the data dictionary, the SQL used, the queries, the user interface, the implementation and the testing.

2. Introduction and Problem Statement

2.1 Introduction

A water tanker service takes booking requests from customers for a delivery address and a time slot, assigns a tanker of the right size and a driver, delivers the water, and collects payment. Each step creates data that must stay consistent. A DBMS gives that consistency through tables, keys, constraints, triggers and transactions, and lets an operator answer questions with SQL.

2.2 Problem Statement

Manual water delivery suffers from four problems, identified in Presentation-I:

- **Phone and paper bookings:** orders are lost, copied wrongly, and there is no single record of demand.
- **No inventory tracking:** tanker availability is not tracked centrally, so vehicles get double-booked.
- **Blind dispatching:** without location data, dispatch is inefficient and customers get no reliable arrival estimate.
- **Cash-only ledgers:** paper payment records cause disputes and make auditing difficult.

The task is to design a normalized relational database and a user interface that record customers, addresses, tankers, drivers, bookings, deliveries and payments reliably, and that block invalid operations such as double-booking at the database layer.

3. Objectives and Scope

3.1 Objectives

- Model the water tanker domain with an Entity-Relationship (ER) diagram in Chen notation.
- Convert the model into a relational schema in Third Normal Form (3NF).
- Implement the schema in MySQL with primary keys, foreign keys, UNIQUE and CHECK constraints.

- Enforce dispatch rules with triggers so they hold whichever frontend is used.
- Build a web UI connected to MySQL that supports insert, delete and view operations.
- Demonstrate joins, aggregates, subqueries, a view, integrity checks and concurrency control.

3.2 Scope

In scope: customers with several delivery addresses, service areas, tanker sizes with prices, the tanker fleet, drivers, time-slot bookings, deliveries, payments, status automation, and revenue analytics.

Out of scope: real GPS hardware (the live radar in the UI is a simulation), online payment gateway integration, and user login with roles.

4. Software and Hardware Requirements

4.1 Software Requirements

Component	Details
Operating system	Windows 10/11
Database	MySQL Community Server (script written for 8.0+, demonstrated in the MySQL 9.7 command-line client)
Database tools	MySQL command-line client, MySQL Workbench
Runtime	Node.js (LTS) with npm
Frontend	React 19, TypeScript, Vite, Tailwind CSS, Motion, Lucide icons
Server layer	Express with dotenv
IDE	PyCharm
Diagramming	draw.io (diagrams.net)
Version control	Git and GitHub
Browser	Google Chrome or Microsoft Edge

4.2 Hardware Requirements

Component	Minimum
Processor	Dual-core, 2 GHz or above
RAM	8 GB
Storage	2 GB free space

Component	Minimum
Display	1366 x 768 or higher

5. ER Diagram

The ER diagram (Chen notation) has nine entities: **CUSTOMER, AREA, ADDRESS, TANKER_TYPE, TANKER, DRIVER, BOOKING, DELIVERY and PAYMENT.**

[Insert Figure 1 here: the ER diagram image, water_tanker_er (Chen notation).png. Caption: ER diagram of the Water Tanker Booking and Delivery Management System]

5.1 Relationships

Relationship	Entities	Cardinality	Meaning
has	CUSTOMER - ADDRESS	1 : N	A customer can register many delivery addresses.
contains	AREA - ADDRESS	1 : N	An area holds many addresses; each address is in one area.
delivered to	ADDRESS - BOOKING	1 : N	An address can receive many bookings.
requested as	TANKER_TYPE - BOOKING	1 : N	A booking asks for one tanker size; a size is requested by many bookings.
classifies	TANKER_TYPE - TANKER	1 : N	One size category covers many tankers.
fulfilled by	BOOKING - DELIVERY	1 : 1	A booking has at most one delivery.
used in	TANKER - DELIVERY	1 : N	A tanker makes many deliveries over time.
drives	DRIVER - DELIVERY	1 : N	A driver makes many deliveries over time.
settled by	BOOKING - PAYMENT	1 : 1	One payment per booking.

6. Relational Schema and Normalization

6.1 Relational Schema

PK = primary key, FK = foreign key.

- **CUSTOMER** (CustomerID PK, Name, Phone)
- **AREA** (AreaID PK, AreaName, Pincode)
- **ADDRESS** (AddressID PK, CustomerID FK, AreaID FK, Street, Latitude, Longitude)
- **TANKER_TYPE** (TypeID PK, Capacity_Liters, Price)
- **TANKER** (TankerID PK, TypeID FK, License_Plate, Status)
- **DRIVER** (DriverID PK, Name, Phone, License_No)
- **BOOKING** (BookingID PK, AddressID FK, TypeID FK, ScheduledDate, TimeSlot, Status)
- **DELIVERY** (DeliveryID PK, BookingID FK, TankerID FK, DriverID FK, DispatchTime, DeliveredTime)
- **PAYMENT** (PaymentID PK, BookingID FK, Amount, Method)

6.2 Normalization

Unnormalized form. A single flat table would repeat customer, address, area, tanker and driver details on every booking row:

BOOKING_ALL (BookingID, CustomerName, CustomerPhone, Street, AreaName, Pincode, Capacity_Liters, Price, LicensePlate, DriverName, DriverLicense, ScheduledDate, TimeSlot, Amount, Method)

This causes update anomalies (a changed phone number must be edited in many rows), insertion anomalies (a new tanker cannot be stored until it is booked) and deletion anomalies (deleting a customer's last booking loses the customer).

1NF. Every column holds one atomic value, there are no repeating groups, and every row has a primary key. All nine tables satisfy this.

2NF. Every non-key attribute depends on the whole key. Each table has a single-column key, so no partial dependency exists. Customer, address, tanker and driver details moved out of the booking record.

3NF. No non-key attribute depends on another non-key attribute. The design choices that achieve this:

- BOOKING has no CustomerID. The customer is reached through ADDRESS (AddressID to CustomerID), so storing it again would create a transitive dependency.
- Price depends on the tanker size, not on the booking, so it lives in TANKER_TYPE next to Capacity_Liters.
- Pincode depends on the area, so it lives in AREA, not in ADDRESS.
- Tanker and driver assignment lives in DELIVERY, so BOOKING stays about what was requested.

7. Data Dictionary

All tables use the InnoDB engine. Foreign keys use ON UPDATE CASCADE and ON DELETE RESTRICT.

7.1 CUSTOMER

Column	Data type	Constraints
CustomerID	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NOT NULL
Phone	VARCHAR(10)	NOT NULL, UNIQUE, CHECK (exactly 10 digits)

7.2 AREA

Column	Data type	Constraints
AreaID	INT	PRIMARY KEY, AUTO_INCREMENT
AreaName	VARCHAR(80)	NOT NULL, UNIQUE
Pincode	CHAR(6)	NOT NULL, CHECK (exactly 6 digits)

7.3 ADDRESS

Column	Data type	Constraints
AddressID	INT	PRIMARY KEY, AUTO_INCREMENT
CustomerID	INT	NOT NULL, FOREIGN KEY to CUSTOMER(CustomerID)
AreaID	INT	NOT NULL, FOREIGN KEY to AREA(AreaID)
Street	VARCHAR(150)	NOT NULL
Latitude	DECIMAL(9,6)	NULL, CHECK (-90 to 90)
Longitude	DECIMAL(9,6)	NULL, CHECK (-180 to 180)

7.4 TANKER_TYPE

Column	Data type	Constraints
TypeID	INT	PRIMARY KEY, AUTO_INCREMENT
Capacity_Liters	INT	NOT NULL, UNIQUE, CHECK (>

Column	Data type	Constraints
		0)
Price	DECIMAL(10,2)	NOT NULL, CHECK (>= 0)

7.5 TANKER

Column	Data type	Constraints
TankerID	INT	PRIMARY KEY, AUTO_INCREMENT
TypeID	INT	NOT NULL, FOREIGN KEY to TANKER_TYPE(TypeID)
License_Plate	VARCHAR(15)	NOT NULL, UNIQUE
Status	VARCHAR(15)	NOT NULL, DEFAULT 'Available', CHECK (Available, Dispatched, Maintenance)

7.6 DRIVER

Column	Data type	Constraints
DriverID	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NOT NULL
Phone	VARCHAR(10)	NOT NULL, CHECK (exactly 10 digits)
License_No	VARCHAR(20)	NOT NULL, UNIQUE

7.7 BOOKING

Column	Data type	Constraints
BookingID	INT	PRIMARY KEY, AUTO_INCREMENT
AddressID	INT	NOT NULL, FOREIGN KEY to ADDRESS(AddressID)
TypeID	INT	NOT NULL, FOREIGN KEY to TANKER_TYPE(TypeID)
ScheduledDate	DATE	NOT NULL

Column	Data type	Constraints
TimeSlot	VARCHAR(11)	NOT NULL, CHECK (06:00-09:00, 09:00-12:00, 12:00-15:00, 15:00-18:00)
Status	VARCHAR(10)	NOT NULL, DEFAULT 'Pending', CHECK (Pending, Assigned, Delivered, Cancelled)

An index, idx_booking_date_status, covers (ScheduledDate, Status).

7.8 DELIVERY

Column	Data type	Constraints
DeliveryID	INT	PRIMARY KEY, AUTO_INCREMENT
BookingID	INT	NOT NULL, UNIQUE, FOREIGN KEY to BOOKING(BookingID)
TankerID	INT	NOT NULL, FOREIGN KEY to TANKER(TankerID)
DriverID	INT	NOT NULL, FOREIGN KEY to DRIVER(DriverID)
DispatchTime	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
DeliveredTime	DATETIME	NULL, CHECK (NULL or not earlier than DispatchTime)

7.9 PAYMENT

Column	Data type	Constraints
PaymentID	INT	PRIMARY KEY, AUTO_INCREMENT
BookingID	INT	NOT NULL, UNIQUE, FOREIGN KEY to BOOKING(BookingID)
Amount	DECIMAL(10,2)	NOT NULL, CHECK (> 0)
Method	VARCHAR(12)	NOT NULL, CHECK (Cash, UPI, Card, NetBanking)

8. SQL Commands Used

The complete script is water_tanker_db.sql. It drops and recreates the database, so running it again resets the sample data.

8.1 DDL: Tables

Parent tables are created first and child tables after them.

```
DROP DATABASE IF EXISTS water_tanker_db;
CREATE DATABASE water_tanker_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE water_tanker_db;
```

```
CREATE TABLE CUSTOMER (
  CustomerID INT AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Phone VARCHAR(10) NOT NULL,
  CONSTRAINT uq_customer_phone UNIQUE (Phone),
  CONSTRAINT chk_customer_phone CHECK (Phone REGEXP '^[0-9]{10}$')
) ENGINE=InnoDB;
```

```
CREATE TABLE AREA (
  AreaID INT AUTO_INCREMENT PRIMARY KEY,
  AreaName VARCHAR(80) NOT NULL,
  Pincode CHAR(6) NOT NULL,
  CONSTRAINT uq_area_name UNIQUE (AreaName),
  CONSTRAINT chk_area_pincode CHECK (Pincode REGEXP '^[0-9]{6}$')
) ENGINE=InnoDB;
```

```
CREATE TABLE ADDRESS (
  AddressID INT AUTO_INCREMENT PRIMARY KEY,
  CustomerID INT NOT NULL,
  AreaID INT NOT NULL,
  Street VARCHAR(150) NOT NULL,
  Latitude DECIMAL(9,6) NULL,
  Longitude DECIMAL(9,6) NULL,
  CONSTRAINT fk_address_customer FOREIGN KEY (CustomerID) REFERENCES CUSTOMER
(CustomerID)
ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_address_area FOREIGN KEY (AreaID) REFERENCES AREA (AreaID)
ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_address_lat CHECK (Latitude BETWEEN -90 AND 90),
  CONSTRAINT chk_address_lon CHECK (Longitude BETWEEN -180 AND 180)
) ENGINE=InnoDB;
```

```
CREATE TABLE TANKER_TYPE (
 TypeID INT AUTO_INCREMENT PRIMARY KEY,
  Capacity_Liters INT NOT NULL,
  Price DECIMAL(10,2) NOT NULL,
  CONSTRAINT uq_type_capacity UNIQUE (Capacity_Liters),
  CONSTRAINT chk_type_capacity CHECK (Capacity_Liters > 0),
  CONSTRAINT chk_type_price CHECK (Price >= 0)
) ENGINE=InnoDB;
```

```
CREATE TABLE TANKER (
```

```

    TankerID INT AUTO_INCREMENT PRIMARY KEY,
    TypeID INT NOT NULL,
    License_Plate VARCHAR(15) NOT NULL,
    Status VARCHAR(15) NOT NULL DEFAULT 'Available',
    CONSTRAINT uq_tanker_plate UNIQUE (License_Plate),
    CONSTRAINT fk_tanker_type FOREIGN KEY (TypeID) REFERENCES TANKER_TYPE (TypeID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_tanker_status CHECK (Status IN
('Available','Dispatched','Maintenance'))
) ENGINE=InnoDB;

CREATE TABLE DRIVER (
    DriverID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(10) NOT NULL,
    License_No VARCHAR(20) NOT NULL,
    CONSTRAINT uq_driver_license UNIQUE (License_No),
    CONSTRAINT chk_driver_phone CHECK (Phone REGEXP '^[0-9]{10}$')
) ENGINE=InnoDB;

CREATE TABLE BOOKING (
    BookingID INT AUTO_INCREMENT PRIMARY KEY,
    AddressID INT NOT NULL,
    TypeID INT NOT NULL,
    ScheduledDate DATE NOT NULL,
    TimeSlot VARCHAR(11) NOT NULL,
    Status VARCHAR(10) NOT NULL DEFAULT 'Pending',
    CONSTRAINT fk_booking_address FOREIGN KEY (AddressID) REFERENCES ADDRESS
(AddressID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_booking_type FOREIGN KEY (TypeID) REFERENCES TANKER_TYPE (TypeID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_booking_slot CHECK (TimeSlot IN
('06:00-09:00','09:00-12:00','12:00-15:00','15:00-18:00')),
    CONSTRAINT chk_booking_status CHECK (Status IN
('Pending','Assigned','Delivered','Cancelled'))
) ENGINE=InnoDB;

CREATE INDEX idx_booking_date_status ON BOOKING (ScheduledDate, Status);

CREATE TABLE DELIVERY (
    DeliveryID INT AUTO_INCREMENT PRIMARY KEY,
    BookingID INT NOT NULL,
    TankerID INT NOT NULL,
    DriverID INT NOT NULL,
    DispatchTime DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    DeliveredTime DATETIME NULL,
    CONSTRAINT uq_delivery_booking UNIQUE (BookingID),
    CONSTRAINT fk_delivery_booking FOREIGN KEY (BookingID) REFERENCES BOOKING
(BookingID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_delivery_tanker FOREIGN KEY (TankerID) REFERENCES TANKER (TankerID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_delivery_driver FOREIGN KEY (DriverID) REFERENCES DRIVER (DriverID)
        ON UPDATE CASCADE ON DELETE RESTRICT,

```

```

    CONSTRAINT chk_delivery_times CHECK (DeliveredTime IS NULL OR DeliveredTime >=
DispatchTime)
) ENGINE=InnoDB;

CREATE TABLE PAYMENT (
    PaymentID INT AUTO_INCREMENT PRIMARY KEY,
    BookingID INT NOT NULL,
    Amount DECIMAL(10,2) NOT NULL,
    Method VARCHAR(12) NOT NULL,
    CONSTRAINT uq_payment_booking UNIQUE (BookingID),
    CONSTRAINT fk_payment_booking FOREIGN KEY (BookingID) REFERENCES BOOKING
(BookingID)
    ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_payment_amount CHECK (Amount > 0),
    CONSTRAINT chk_payment_method CHECK (Method IN ('Cash','UPI','Card','NetBanking'))
) ENGINE=InnoDB;

```

8.2 Triggers

Three triggers on DELIVERY keep the dispatch rules inside the database, so they hold no matter which application writes to it.

```

DELIMITER //

CREATE TRIGGER trg_delivery_before_insert
BEFORE INSERT ON DELIVERY FOR EACH ROW
BEGIN
    DECLARE v_booking_status VARCHAR(10);
    DECLARE v_booking_type INT;
    DECLARE v_tanker_status VARCHAR(15);
    DECLARE v_tanker_type INT;
    DECLARE v_driver_busy INT;

    SELECT Status, TypeID INTO v_booking_status, v_booking_type
    FROM BOOKING WHERE BookingID = NEW.BookingID;
    SELECT Status, TypeID INTO v_tanker_status, v_tanker_type
    FROM TANKER WHERE TankerID = NEW.TankerID;
    SELECT COUNT(*) INTO v_driver_busy
    FROM DELIVERY WHERE DriverID = NEW.DriverID AND DeliveredTime IS NULL;

    IF v_booking_status IS NULL OR v_booking_status <> 'Pending' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Booking not found or not in Pending
status';
    END IF;
    IF v_tanker_status IS NULL OR v_tanker_status <> 'Available' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tanker is not available';
    END IF;
    IF v_tanker_type <> v_booking_type THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tanker size does not match the
booking';
    END IF;
    IF v_driver_busy > 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Driver is already on another
delivery';
    END IF;

```

```

END//

CREATE TRIGGER trg_delivery_after_insert
AFTER INSERT ON DELIVERY FOR EACH ROW
BEGIN
    IF NEW.DeliveredTime IS NULL THEN
        UPDATE TANKER SET Status = 'Dispatched' WHERE TankerID = NEW.TankerID;
        UPDATE BOOKING SET Status = 'Assigned' WHERE BookingID = NEW.BookingID;
    ELSE
        UPDATE BOOKING SET Status = 'Delivered' WHERE BookingID = NEW.BookingID;
    END IF;
END//

CREATE TRIGGER trg_delivery_after_update
AFTER UPDATE ON DELIVERY FOR EACH ROW
BEGIN
    IF OLD.DeliveredTime IS NULL AND NEW.DeliveredTime IS NOT NULL THEN
        UPDATE BOOKING SET Status = 'Delivered' WHERE BookingID = NEW.BookingID;
        UPDATE TANKER SET Status = 'Available' WHERE TankerID = NEW.TankerID;
    END IF;
END//

DELIMITER ;

```

8.3 View

The view vw_booking_details returns one flat row per booking, joining customer, address, area, tanker size, delivery, tanker, driver and payment. The UI and the demo queries read from it.

```

CREATE VIEW vw_booking_details AS
SELECT b.BookingID, c.CustomerID, c.Name AS CustomerName, c.Phone AS CustomerPhone,
       a.Street, ar.AreaName, ar.Pincode, tt.Capacity_Liters, tt.Price,
       b.ScheduledDate, b.TimeSlot, b.Status AS BookingStatus,
       d.DeliveryID, t.License_Plate, dr.Name AS DriverName,
       d.DispatchTime, d.DeliveredTime,
       p.Amount AS PaidAmount, p.Method AS PaymentMethod
FROM BOOKING b
JOIN ADDRESS a      ON a.AddressID = b.AddressID
JOIN CUSTOMER c     ON c.CustomerID = a.CustomerID
JOIN AREA ar        ON ar.AreaID    = a.AreaID
JOIN TANKER_TYPE tt ON tt.TypeID     = b.TypeID
LEFT JOIN DELIVERY d ON d.BookingID = b.BookingID
LEFT JOIN TANKER t   ON t.TankerID  = d.TankerID
LEFT JOIN DRIVER dr  ON dr.DriverID  = d.DriverID
LEFT JOIN PAYMENT p  ON p.BookingID = b.BookingID;

```

8.4 DML: Sample Data

The script loads 6 areas, 8 customers, 10 addresses, 3 tanker sizes, 6 tankers, 4 drivers, 12 bookings, 8 deliveries and 7 payments. Parents are inserted before children. Representative statements:

```

INSERT INTO AREA (AreaName, Pincode) VALUES ('Gandhi Nagar', '500101'), ('Lakshmi
Nagar', '500102');
INSERT INTO CUSTOMER (Name, Phone) VALUES ('Ravi Kumar', '9000000001'), ('Sneha
Reddy', '9000000002');

```

```

INSERT INTO ADDRESS (CustomerID, AreaID, Street, Latitude, Longitude)
VALUES (1, 1, '12-3-45, Main Road', 17.401200, 78.480100);
INSERT INTO TANKER_TYPE (Capacity_Liters, Price) VALUES (3000, 600.00), (6000,
1000.00), (10000, 1500.00);
INSERT INTO TANKER (TypeID, License_Plate, Status) VALUES (1, 'TS01WT1001',
'Available');
INSERT INTO DRIVER (Name, Phone, License_No) VALUES ('Ramesh Goud', '9100000001',
'DL-2026-0001');
INSERT INTO BOOKING (AddressID, TypeID, ScheduledDate, TimeSlot, Status)
VALUES (1, 2, '2026-09-16', '06:00-09:00', 'Pending');
INSERT INTO DELIVERY (BookingID, TankerID, DriverID, DispatchTime, DeliveredTime)
VALUES (1, 3, 1, '2026-09-16 06:30:00', '2026-09-16 07:40:00');
INSERT INTO PAYMENT (BookingID, Amount, Method) VALUES (1, 1000.00, 'UPI');
Bookings are loaded as Pending. The delivery triggers then move them to Assigned or Delivered by
themselves.

```

8.5 Sample Outputs

SHOW TABLES lists the nine tables and the view:

```

mysql> use water_tanker_db;
Database changed
mysql> show tables;
+-----+
| Tables_in_water_tanker_db |
+-----+
| address                    |
| area                      |
| booking                   |
| customer                  |
| delivery                  |
| driver                    |
| payment                   |
| tanker                    |
| tanker_type               |
| vw_booking_details        |
+-----+
10 rows in set

```

SELECT * FROM customer:

CustomerID	Name	Phone
1	Ravi Kumar	9000000001
2	Sneha Reddy	9000000002
3	Anil Sharma	9000000003
4	Lakshmi Devi	9000000004
5	Mohammed Imran	9000000005
6	Priya Nair	9000000006
7	Suresh Babu	9000000007
8	Kavitha Rao	9000000008

8 rows in set

[Insert Figures 2 to 4 here: the MySQL command-line screenshots of SHOW TABLES, SELECT * FROM customer, and SELECT * FROM vw_booking_details.]

9. Queries with Outputs

All queries are in demo_queries.sql and run after water_tanker_db.sql. Output depends on the data at the time of the run, so paste a screenshot of your own run under each query.

9.1 Pending bookings per area (3-table join, GROUP BY)

```
SELECT ar.AreaName, COUNT(*) AS pending_bookings
FROM BOOKING b
JOIN ADDRESS a ON a.AddressID = b.AddressID
JOIN AREA ar   ON ar.AreaID   = a.AreaID
WHERE b.Status = 'Pending'
GROUP BY ar.AreaName
ORDER BY pending_bookings DESC;
```

[Output screenshot]

9.2 Every booking with customer, area and tanker size (5-table join)

```
SELECT b.BookingID, c.Name AS Customer, ar.AreaName, tt.Capacity_Liters,
       b.ScheduledDate, b.TimeSlot, b.Status
FROM BOOKING b
JOIN ADDRESS a      ON a.AddressID = b.AddressID
JOIN CUSTOMER c     ON c.CustomerID = a.CustomerID
JOIN AREA ar        ON ar.AreaID   = a.AreaID
JOIN TANKER_TYPE tt ON tt.TypeID    = b.TypeID
ORDER BY b.ScheduledDate, b.TimeSlot;
```

[Output screenshot]

9.3 Revenue by tanker size (aggregate)

```
SELECT tt.Capacity_Liters, COUNT(p.PaymentID) AS payments, SUM(p.Amount) AS revenue
FROM PAYMENT p
JOIN BOOKING b      ON b.BookingID = p.BookingID
JOIN TANKER_TYPE tt ON tt.TypeID    = b.TypeID
GROUP BY tt.Capacity_Liters
ORDER BY revenue DESC;
```

[Output screenshot]

9.4 Trips per tanker, including tankers with none (LEFT JOIN)

```
SELECT t.License_Plate, t.Status, COUNT(d.DeliveryID) AS trips
FROM TANKER t
LEFT JOIN DELIVERY d ON d.TankerID = t.TankerID
GROUP BY t.TankerID, t.License_Plate, t.Status
ORDER BY trips DESC;
```

[Output screenshot]

9.5 Customers with more than one booking (GROUP BY and HAVING)

```
SELECT c.Name, COUNT(b.BookingID) AS total_bookings
FROM CUSTOMER c
```

```

JOIN ADDRESS a ON a.CustomerID = c.CustomerID
JOIN BOOKING b ON b.AddressID = a.AddressID
GROUP BY c.CustomerID, c.Name
HAVING COUNT(b.BookingID) > 1;

```

[Output screenshot]

9.6 Tankers that can serve pending booking 9 (subquery)

```

SELECT t.TankerID, t.License_Plate
FROM TANKER t
WHERE t.Status = 'Available'
AND t.TypeID = (SELECT TypeID FROM BOOKING WHERE BookingID = 9);

```

[Output screenshot]

9.7 Average delivery time per driver in minutes (date function)

```

SELECT dr.Name, COUNT(*) AS completed_trips,
       ROUND(AVG(TIMESTAMPDIFF(MINUTE, d.DispatchTime, d.DeliveredTime))) AS
avg_minutes
FROM DELIVERY d
JOIN DRIVER dr ON dr.DriverID = d.DriverID
WHERE d.DeliveredTime IS NOT NULL
GROUP BY dr.DriverID, dr.Name;

```

[Output screenshot]

9.8 Deliveries currently on the road (uses the view)

```

SELECT BookingID, CustomerName, AreaName, License_Plate, DriverName, DispatchTime
FROM vw_booking_details
WHERE BookingStatus = 'Assigned';

```

[Output screenshot]

9.9 Trigger demonstration

Assigning a tanker and driver is one INSERT. The triggers change the booking and tanker statuses with no manual update.

```

-- Before: booking 9 is Pending, tanker 2 is Available
SELECT BookingID, Status FROM BOOKING WHERE BookingID = 9;
SELECT TankerID, Status FROM TANKER WHERE TankerID = 2;

-- Assign tanker 2 and driver 3 to booking 9
INSERT INTO DELIVERY (BookingID, TankerID, DriverID) VALUES (9, 2, 3);
-- After: booking 9 is Assigned, tanker 2 is Dispatched

-- Complete the delivery
UPDATE DELIVERY SET DeliveredTime = NOW() WHERE BookingID = 9;
-- Booking 9 is Delivered again, tanker 2 is Available again

```

[Output screenshots: the SELECTs before and after the INSERT and after the UPDATE]

9.10 Concurrency demonstration

Two query tabs prove that a tanker row is locked while one transaction works on it. Tab 2 waits until Tab 1 commits.

```
-- Tab 1
START TRANSACTION;
SELECT * FROM TANKER WHERE TankerID = 4 AND Status = 'Available' FOR UPDATE;
-- run the same SELECT ... FOR UPDATE in Tab 2: it waits
COMMIT; -- Tab 2 now continues
```

9.11 Query given during Presentation-II

[Paste the exact question you were given in Presentation-II, your SQL solution, and a screenshot of its output here. This section is required by the guidelines.]

10. UI Design and Screenshots

AquaFlow is a dark-themed web dashboard for the dispatcher, with a top bar of live counters (tankers in transit, available fleet) and a navigation row: Dashboard, Live GPS Radar, Active Deliveries, Fleet and Inventory, Bookings, Customers, Database Studio (9 tables) and Analytics. Every screen reads from MySQL, and every change is saved to MySQL and shown again after a refresh.

10.1 The three required operations

Operation	Where in the UI	What happens in the database
Insert	New Booking, Add New Customer, Dispatch Tanker	New rows in BOOKING, CUSTOMER and ADDRESS, or DELIVERY (the triggers then set Assigned and Dispatched)
Delete	Trash icon on each booking and customer row; Abort on an assigned booking	The row is removed; a delete that would break a foreign key is refused and the data stays intact
View	Dashboard, Bookings registry, Customer directory, Fleet and Inventory, Active Deliveries, Analytics, Database Studio	SELECT queries over the tables and the vw_booking_details view

Each booking and customer row also has an edit icon.

10.2 Screens

Dashboard. Four live cards: tankers in transit, tankers available at the depot out of six, pending dispatch orders, and total receipts (7,200 rupees in the captured run). Below them are the in-transit tanker with its driver, route progress and a Confirm Delivery button, and the pending orders queue with a Dispatch button and a delete icon on each order.

[Insert Figure 5 here: Dashboard screenshot]

Bookings registry. A table of every booking with booking ID, customer and phone, address and area with pincode, capacity and rate, date and slot, status, assigned tanker and driver, and row actions. A search

box and Status and Time Slot filters narrow the list. The captured run shows bookings #13 to #15 created through the UI, with #15 in Assigned status on tanker TS01WT1004.

[Insert Figure 6 here: Bookings screenshot]

Customer directory. Summary cards (11 customers, 15 bookings, 10 completed trips, 10,200 rupees of delivered inflow), a search box, and one row per customer with phone, registered addresses, booking count, a Book Tanker button, and edit and delete icons.

[Insert Figure 7 here: Customers screenshot]

Analytics. A 30-day cumulative revenue curve with the average daily run-rate, the best single day and the number of active inflow days, with a switch to daily revenue inflow.

[Insert Figure 8 here: Analytics screenshot]

[Insert before and after screenshots for the insert and delete operations here: a booking or customer before it is added or deleted, the screen after it, and the matching SELECT in MySQL.]

11. Implementation Details

11.1 Technology Stack

Layer	Technology
Database	MySQL, InnoDB engine, triggers, view, CHECK constraints
Frontend	React 19, TypeScript, Vite, Tailwind CSS, Motion, Lucide icons
Server layer	Node.js with Express and dotenv
Tools	MySQL command-line client, MySQL Workbench, PyCharm, Git and GitHub

11.2 Architecture

A browser cannot talk to MySQL directly, so the React UI calls a small Express server, and the server runs SQL against MySQL:

React UI (browser) -> Express server (Node.js) -> MySQL database (water_tanker_db)

11.3 Where the Rules Live

The dispatch rules are in the database, not in the UI code. Dispatching a tanker is one statement:

```
INSERT INTO DELIVERY (BookingID, TankerID, DriverID) VALUES (9, 2, 3);
```

The BEFORE INSERT trigger rejects the statement with a clear message if the booking is not Pending, the tanker is not Available, the tanker size does not match the booking, or the driver is already on a delivery. The AFTER INSERT trigger then marks the booking Assigned and the tanker Dispatched. The AFTER

UPDATE trigger closes the booking and frees the tanker when DeliveredTime is filled in. The UI shows the trigger's message when a dispatch is refused. This protects the data even if another program writes to the database.

Other integrity features: ON DELETE RESTRICT on every foreign key, UNIQUE on one delivery and one payment per booking, CHECK constraints on phone, pincode, coordinates, time slot, statuses, amounts and delivery times, and row locking with SELECT ... FOR UPDATE for concurrent dispatchers.

11.4 Database Connection Code

[Paste the connection code from your server file here: host, user, database name, and how the password is read from the .env file. Add one or two route handlers that show a view, an insert and a delete.]

12. Testing

The cases below come from the demo script (demo_queries.sql) and the UI. Fill in the Result column after each run, and attach a screenshot for the important ones.

ID	Test case	Action	Expected result	Result
T1	Load the schema	Run water_tanker_db.sql	9 tables and the view vw_booking_details exist	Pass (Figure 2)
T2	View data in SQL	SELECT * FROM customer	8 sample customers listed	Pass (Figure 3)
T3	View in the UI	Open the Customers and Bookings pages	Rows match the database	Pass (Figures 6, 7)
T4	Insert through the UI	Create a booking in New Booking	New row appears in BOOKING and in the registry	Pass (bookings #13 to #15, Figure 6)
T5	Dispatch	Assign a tanker and driver to a Pending booking	Booking becomes Assigned, tanker becomes Dispatched	Pass (booking #15, Figure 6)
T6	Complete a delivery	Confirm Delivery, or set DeliveredTime	Booking becomes Delivered, tanker becomes Available	[Pass/Fail]
T7	Delete through the UI	Delete a booking or customer with	Row removed from the screen	[Pass/Fail]

ID	Test case	Action	Expected result	Result
		no dependent rows	and from MySQL	
T8	Tanker in the workshop	INSERT INTO DELIVERY (10, 6, 3)	Rejected: Tanker is not available	[Pass/Fail]
T9	Double-booking	INSERT INTO DELIVERY (10, 1, 3) while tanker 1 is on the road	Rejected: Tanker is not available	[Pass/Fail]
T10	Wrong tanker size	INSERT INTO DELIVERY (10, 4, 3)	Rejected: Tanker size does not match the booking	[Pass/Fail]
T11	Busy driver	INSERT INTO DELIVERY (10, 5, 1) while driver 1 is on a delivery	Rejected: Driver is already on another delivery	[Pass/Fail]
T12	Booking already delivered	INSERT INTO DELIVERY (1, 2, 3)	Rejected: Booking not found or not in Pending status	[Pass/Fail]
T13	Foreign key	INSERT a booking with AddressID 999	Rejected by the foreign key	[Pass/Fail]
T14	CHECK constraint	INSERT a booking with time slot 23:00-24:00	Rejected by the CHECK constraint	[Pass/Fail]
T15	Referential integrity	DELETE the customer with ID 1	Blocked by ON DELETE RESTRICT	[Pass/Fail]
T16	UNIQUE constraint	Insert a second payment for booking 1	Rejected: one payment per booking	[Pass/Fail]
T17	Concurrency	Run SELECT ... FOR UPDATE on tanker 4 in two tabs	Tab 2 waits until Tab 1 commits	[Pass/Fail]

13. Conclusion and Future Enhancements

13.1 Conclusion

The project replaces phone calls and paper ledgers with a normalized MySQL database and the AquaFlow web application. The nine-table schema is in Third Normal Form, and the triggers, constraints and locking keep the data valid however it is changed. The UI supports insert, delete and view operations, tanker dispatch with automatic status changes, and revenue analytics, and every change is reflected in the database.

13.2 Future Enhancements

- Real GPS tracking from tanker devices, replacing the simulated live radar.
- Online payment gateway integration for UPI and card payments.
- Login with roles for admin, dispatcher and driver.
- Customer mobile app with SMS or WhatsApp booking alerts.
- Automatic tanker and driver suggestion by area and availability.
- Cloud deployment with scheduled database backups.

14. References

1. Elmasri, R. and Navathe, S. B., *Fundamentals of Database Systems*, Pearson.
2. Silberschatz, A., Korth, H. F. and Sudarshan, S., *Database System Concepts*, McGraw-Hill.
3. MySQL Reference Manual, <https://dev.mysql.com/doc/>
4. Node.js Documentation, <https://nodejs.org/en/docs>
5. Express Documentation, <https://expressjs.com>
6. React Documentation, <https://react.dev>
7. Tailwind CSS Documentation, <https://tailwindcss.com/docs>
8. draw.io (diagrams.net), <https://www.diagrams.net>
9. DBMS course lecture notes, Woxsen University.

15. Appendix: GitHub Repository Link

Repository: <https://github.com/CodeInfinity08/DBMS-Course-Project>

The repository holds the folders Presentation-I, Presentation-II, Presentation-III and Project-Report, with a README.md.